

Übungsszenario: Linux Command Line Basics

Dauer: 30 Minuten

Hilfsmittel: `man`, `--help` erlaubt

System: Linux mit Bash-Shell

Ausgangssituation

Du arbeitest als **IT-Techniker in Ausbildung**.

Für eine Schulung sollst du eine kleine **Log-Auswertung** über die Kommandozeile durchführen.

Lernziele

Dabei sollst du zeigen, dass du:

- Dateien strukturieren kannst
- Inhalte erzeugen und auswerten kannst
- effizient mit **Pipes und Filtern** arbeitest
- Variablen in Workflows integrierst

Aufgaben

Aufgabe 1 – Verzeichnis & Dateien (2 Punkte)

1. Wechsle in dein Home-Verzeichnis
2. Erstelle ein neues Verzeichnis: `htl_log_analysis`
3. Wechsle in dieses Verzeichnis
4. Erstelle folgende Dateien:
 - `access.log`
 - `error.log`
 - `summary.txt`
 - `backup data.log`

Hinweis: Achte auf korrektes Quoting

Aufgabe 2 – Testdaten erzeugen (2 Punkte)

1. Schreibe mit **einem Befehl** mehrere Zeilen in `access.log`, z. B.:

```
user1 OK
user2 ERROR
admin OK
guest ERROR
```

2. Verwende dafür `echo -e`

3. Ergänze zusätzlich deinen aktuellen Benutzer (`whoami`) in der Datei

Hinweis: Kombination von Text + Command Substitution

Aufgabe 3 – Filtern (1 Punkt)

1. Filtere aus `access.log`:
 - nur Zeilen mit **ERROR**
2. Sortiere diese alphabetisch
3. Speichere das Ergebnis in `error.log`

Hinweis: Verwende **Pipes**.

Aufgabe 4 – Auswerten (1 Punkt)

1. Zähle die Anzahl der Fehler
2. Gib das Ergebnis in folgendem Format aus:

Gefundene Fehler: X

Hinweis: Verwende **Command Substitution**.

`$(...)` bindet Ergebnis in Text ein

Aufgabe 5 – Erweiterte Analyse (2 Punkte)

- Zeige den Inhalt von `error.log` **seitenweise**
- Nummeriere zusätzlich die Zeilen
- Zeige nur die **erste Zeile** der Datei

Aufgabe 6 - Verständnisfragen (2 Punkte)

Frage 1 – Effizienz von Shell-Befehlen

Ein Benutzer verwendet folgenden Ablauf:

```
cat access.log > temp.txt
grep ERROR temp.txt > result.txt
sort result.txt > sorted.txt
wc -l sorted.txt
```

Erkläre, warum dieser Ansatz ineffizient ist und wie man ihn mithilfe von Pipes verbessern kann. Gehe dabei insbesondere auf folgende Punkte ein:

- unnötige Zwischendateien
- Auswirkungen auf Performance und Übersichtlichkeit
- Vorteil eines direkten Datenstroms

Frage 2 – Variablen

Erkläre, warum Variablen in der Bash hilfreich sind.

Gehe dabei darauf ein:

- wie sie die Wiederverwendbarkeit von Befehlen verbessern
- welchen Vorteil sie bei längeren oder komplexen Dateinamen haben

Typische Fehler

- Vergessen von Quotes bei Dateien mit Leerzeichen
- `grep` ohne korrektes Pattern
- unnötige Zwischendateien statt Pipes
- Überschreiben von Dateien durch `>`

Linux Command Line – Befehlsübersicht

Kategorie	Befehl	Zweck	Hinweis
Shell & Orientierung	<code>whoami</code>	Aktuellen Benutzer anzeigen	Keine Parameter nötig
Shell & Orientierung	<code>pwd</code>	Aktuelles Verzeichnis anzeigen	Wichtig bei relativen Pfaden
Navigation	<code>cd</code>	Verzeichnis wechseln	<code>cd ~</code> → Home
Dateien	<code>ls</code>	Verzeichnisinhalt anzeigen	Mit Optionen kombinierbar
Dateien	<code>mkdir</code>	Verzeichnis erstellen	Mehrere möglich
Dateien	<code>touch</code>	Leere Datei erstellen	Auch mehrere
Dateien	<code>cat</code>	Dateiinhalt ausgeben	Für kurze Dateien
Dateien	<code>less</code>	Datei seitenweise anzeigen	Mit <code>q</code> beenden
Filtern	<code>grep</code>	Text suchen	Groß/Kleinschreibung beachten
Analyse	<code>wc -l</code>	Zeilen zählen	Oft mit Pipe
Sortieren	<code>sort</code>	Alphabetisch sortieren	Textbasiert
Analyse	<code>n1</code>	Zeilen nummerieren	Gut für Auswertung
Analyse	<code>head</code>	Erste Zeilen anzeigen	<code>-n</code> Anzahl
Analyse	<code>tail</code>	Letzte Zeilen anzeigen	<code>-n</code> Anzahl
Hilfe	<code>man</code>	Handbuch anzeigen	Prüfungstauglich
Hilfe	<code>--help</code>	Kurzhilfe	Kurz & kompakt

Merkhilfe

Konzept	Merksatz
Syntax	Befehl → Option → Argument
Variablen	Ohne <code>\$</code> kein Inhalt
Quoting	Leerzeichen = Quotes
Pipes	Ausgabe wird weitergereicht
Prüfung	Sauber & strukturiert arbeiten